llm-verifier_README

LLM Verifier

LLMsVerifier Logo

Verify. Monitor. Optimize.

LLM Verifier is a comprehensive tool to verify, test, and benchmark LLMs based on their coding capabilities and other features. It supports OpenAI-compatible APIs and provides detailed analysis of model capabilities.

Supported Providers (12 Implemented Adapters)

- **OpenAI**: GPT models with full API compatibility
- **Anthropic**: Claude models via official API
- **DeepSeek**: DeepSeek models with streaming support
- **Groq**: Fast inference with Llama models
- **Together AI**: Wide range of open-source models
- **Mistral**: European provider with advanced models
- **xAI**: Grok models from xAI
- **Replicate**: Model hosting and deployment platform
- **Cohere**: Command models with enterprise features
- **Cerebras**: High-performance inference
- **Cloudflare Workers AI**: Edge AI inference
- **SiliconFlow**: Chinese AI models and services

Features

- **Model Discovery**: Automatically discover all available models from API endpoints
- **Comprehensive Testing**: Test model existence, responsiveness, overload status, and capabilities
- **Feature Detection**: Identify supported features like tool calling, embeddings, code generation, etc.
- **Coding Assessment**: Evaluate coding capabilities across multiple programming languages
- **Performance Scoring**: Calculate detailed scores for code capability, responsiveness, reliability, and feature richness
- **Reporting**: Generate both human-readable markdown reports and machine-readable JSON reports
- **Rankings**: Sort models by various criteria (strength, speed, reliability, etc.)

Installation

```
# Clone the repository
git clone <repository-url>
cd llm-verifier
```

```
# Build the application
go build -o llm-verifier cmd/main.go
```

Usage

Command Line Interface

The LLM Verifier provides a comprehensive CLI with multiple subcommands for managing models, providers, verification results, and system configuration.

Global Options

```
-c, --config string     Configuration file path (default
        "config.yaml")
-s, --server string     API server URL (default "http://
        localhost:8080")
-u, --username string   Username for authentication
-p, --password string   Password for authentication
-o, --output string     Output directory for reports (default "./
        reports")
```

Available Commands

Models Management

```
# List all models
llm-verifier models list [--filter NAME] [--limit N] [--format
        json|table]

# Get specific model details
llm-verifier models get MODEL_ID

# Create a new model
llm-verifier models create PROVIDER_ID MODEL_ID NAME

# Verify a specific model
llm-verifier models verify MODEL_ID
```

Examples:

```
# List all models in table format
llm-verifier models list --format table

# Filter models by name
llm-verifier models list --filter gpt

# Get details of a specific model
llm-verifier models get 123
```

```
# Create a new model
llm-verifier models create 1 gpt-4-turbo "GPT-4 Turbo"
```

Providers Management

```
# List all providers
llm-
        verifier providers list [--filter NAME] [--limit N] [--
        format json|table]

# Get specific provider details
llm-verifier providers get PROVIDER_ID
```

Examples:

```
# List providers with filtering
llm-verifier providers list --filter openai --format table

# Get provider details
llm-verifier providers get 1
```

Verification Results

```
# List all verification results
llm-verifier results list [--filter MODEL_NAME] [--limit N] [--
        format json|table]

# Get specific result details
llm-verifier results get RESULT_ID
```

Examples:

```
# List recent results
llm-verifier results list --limit 10 --format table

# Filter results by model
llm-verifier results list --filter gpt-4
```

Pricing Information

```
# List all pricing entries
llm-verifier pricing list [--format json|table]
```

Rate Limits

```
# List all rate limit entries
llm-verifier limits list [--format json|table]
```

Issues Tracking

```
# List all reported issues
llm-verifier issues list [--format json|table]
```

System Events

```
# List all system events
llm-verifier events list [--format json|table]
```

Verification Schedules

```
# List all schedules
llm-verifier schedules list [--format json|table]
```

Configuration Exports

```
# List all configuration exports
llm-verifier exports list [--format json|table]

# Download a specific export
llm-verifier exports download EXPORT_ID
```

System Logs

```
# List system logs
llm-verifier logs list [--format json|table]
```

Configuration Management

```
# Show current configuration
llm-verifier config show

# Export configuration in different formats
llm-verifier config export FORMAT
```

User Management

```
# Create a new user
llm-verifier users create USERNAME PASSWORD EMAIL [FULL_NAME]
```

Terminal User Interface

```
# Start the interactive TUI
llm-verifier tui
```

TUI Features: - Real-time dashboard with statistics - Model browser with filtering and sorting - Provider management interface - Verification results viewer - Auto-refresh every 30-60 seconds - Keyboard navigation (arrow keys, numbers 1-4) - Progress indicators and visual feedback

TUI Navigation

Screen Navigation: - `1` - Dashboard (statistics and overview) - `2` - Models (browse and manage models) - `3` - Providers (manage LLM providers) - `4` - Verification (view verification results) - `←`/`→` or `h`/`l` - Navigate between screens - `q` or `Ctrl+C` - Quit application

Dashboard Screen: - `r` or `R` - Manual refresh of statistics - Auto-refreshes every 30 seconds - Shows verification progress, average scores, and system status

Models Screen: - `↑`/`↓` or `k`/`j` - Navigate through model list - `Enter` or `Space` - Verify selected model - `r` or `R` - Refresh model list - Auto-refreshes every 60 seconds - Shows verification status, scores, and capabilities

Providers Screen: - `↑`/`↓` or `k`/`j` - Navigate through provider list - `Enter` or `Space` - Toggle provider status - `a` or `A` - Add API key for selected provider - `r` or `R` - Refresh provider list - Auto-refreshes every 60 seconds

Verification Screen: - `↑`/`↓` or `k`/`j` - Navigate through results - Shows verification history and status

Visual Indicators: - ● - Pending/Inactive (gray) - ✓ - Verified/Active (green) - ✗ - Failed/Error (red) - Progress bars show completion percentages - Star ratings indicate score quality - Color-coded status indicators throughout

REST API Server

```
# Start the REST API server
llm-verifier server [--port PORT]
```

Basic Usage

```
./llm-verifier
```

This will use the default configuration file `config.yaml` and output reports to the `./reports` directory.

With Custom Configuration

```
./llm-verifier -c /path/to/config.yaml -o /path/to/output
```

Output Formats

All list commands support multiple output formats:

- **JSON** (default): Machine-readable structured data
- **Table**: Human-readable formatted tables with columns

```
# JSON output (default)
llm-verifier models list

# Table output
llm-verifier models list --format table
```

```
# Filtered results
llm-verifier models list --filter gpt --limit 5 --format table
```

Configuration

The tool uses a YAML configuration file. See `config.yaml` for a sample configuration:

```yaml
global:
  base_url: "https://api.openai.com/v1"
  api_key: "${OPENAI_API_KEY}" # Use environment variable
  max_retries: 3
  request_delay: 1s
  timeout: 30s

# If no LLMs are specified, the tool will automatically discover
        all available models
# llms: # Uncomment this section to specify specific LLMs to test
#   - name: "OpenAI GPT-4"
#     endpoint: "https://api.openai.com/v1"
#     api_key: "${OPENAI_API_KEY}"
#     model: "gpt-4-turbo"
#     headers:
#       Custom-Header: "value"
#     features:
#       tool_calling: true
#       embeddings: false
#
#   - name: "OpenAI GPT-3.5"
#     endpoint: "https://api.openai.com/v1"
#     api_key: "${OPENAI_API_KEY}"
#     model: "gpt-3.5-turbo"
#     features:
#       tool_calling: true
#       embeddings: false

concurrency: 5
timeout: 60s
```

Output

The tool generates two types of reports:

1. **Markdown Report** (`llm_verification_report.md`): Human-readable report with detailed analysis
2. **JSON Report** (`llm_verification_report.json`): Machine-readable report for programmatic use

Test Suite

The project includes comprehensive tests:

- Unit tests: Test individual functions and scoring algorithms
- Integration tests: Test component interactions
- End-to-end tests: Test complete workflows
- Performance tests: Benchmark critical functions
- Security tests: Verify secure handling of sensitive data
- Automation tests: Test automated workflows

Run all tests with:

```
go test ./tests/... -v
```

Architecture

The tool is organized into several packages:

- `cmd/`: Main application entry point
- `llmverifier/`: Core verification logic
- `config/`: Configuration structures and loading
- `reports/`: Report generation
- `tests/`: Comprehensive test suite

Capabilities Tested

The tool evaluates models across several dimensions:

- **Code Generation**: Ability to write code in multiple languages
- **Code Completion**: Ability to complete partial code
- **Code Debugging**: Ability to identify and fix code issues
- **Code Review**: Ability to review and improve code
- **Code Explanation**: Ability to explain code functionality
- **Test Generation**: Ability to create unit tests
- **Documentation**: Ability to create documentation
- **Refactoring**: Ability to improve code structure
- **Architecture Understanding**: Ability to design system architectures
- **Security Assessment**: Ability to identify security issues
- **Pattern Recognition**: Ability to implement design patterns
- **Language-Specific Tests**: Performance across multiple programming languages

Scoring System

Models are scored across multiple dimensions:

- **Code Capability (40%)**: How well the model handles coding tasks
- **Responsiveness (20%)**: Response time and throughput
- **Reliability (20%)**: Availability and consistency
- **Feature Richness (15%)**: Supported features and capabilities
- **Value Proposition (5%)**: Overall value for coding tasks

License

MIT